

GPU Cluster Scheduling via Checkpoint-based Live Migration

Jiaxi Shi^{*} Jianchen Hu^{†*} Meng Zhang^{*} Chengshuai Wu^{*}

^{*} *Department of Automation, School of Electronic and Information Engineering, Xi'an Jiaotong University, Xi'an, 710049, China (e-mail: horace89@xjtu.edu.cn).*

Abstract: The proliferation of training requirements for generative artificial intelligence (AI) has made the graphic processing unit (GPU) scheduling a critical issue in cloud computing. Due to the existence of uncertainty in the GPU demand sequence, a poorly designed GPU scheduling algorithm can result in severe GPU fragmentation, leading to low resource utilization ratio and increased operational cost. In order to solve this problem, we propose a new scheduling algorithm that uses the intrinsic checkpointing mechanism of AI training tasks to enable live task migration, so that our approach can consolidate fragmented GPU resources through rescheduling tasks. Moreover, we introduce a dual-objective scheduling strategy tailored to cluster workload which minimizes the average fragmentation rate for dense workloads and minimizes the task queueing time for sparse workloads. We have verified the improvement of our algorithm in cluster throughput and efficiency through a GPU scheduling simulation with 50 servers, each contained 8 GPUs.

Keywords: Cloud computing, generative AI, GPU scheduling, live migration.

1. INTRODUCTION

With the proliferation of training requirements for generative artificial intelligence (AI), especially large language models (LLMs), the demand for GPU resource has arisen increasingly. Utilizing the shared GPU clusters as a cloud resource have emerged as the standard procedure for training AI models for individual users since it can provide low infrastructure cost and elastic resource. In this case, the profit for the cloud provider depends on how they manage the GPU resources. The main objective is to maximize the GPU resource utilization ratio to reduce the operational and holding costs. It is known that reducing the fragment ratio in cloud computing with dynamic and uncertain demands is a challenging task (even for virtual machine (VM) placement, see Hu et al. (2025)). The traditional BestFit/FirstFit bin packing algorithms still are the most popular scheduling algorithms in cloud computing due to their simplicity and guaranteed competitive ratio (Hadary et al., 2020).

Unlike the case with VM placement where the fragment cannot be utilized, the fragmentation manifests as a scattering of idle GPUs across multiple servers for GPU clusters. Even though the fragment GPU resources can be utilized in aggregation, it cannot be consolidated to satisfy the contiguous allocation requirements of incoming multi-GPU training jobs. This can result in reduced GPU utilization ratio, prolonged job queueing times and increased unit cost. The existing scheduling approaches usually suffer from the imbalance between efficiency and optimality. In fact, in order to satisfy the real-time requirement in cloud computing, many algorithms are myopic, meaning that they usually focus on the immediate return but ignores

the long-term topological health of the cluster (such as AnyFit-type algorithms). On the other hand, some other approaches that try to integrate the fragment resources may actually exacerbate internal fragmentation (this will be demonstrated in Section 3).

In order to address these limitations, we introduce a new scheduling framework that utilizes a checkpoint-based migration mechanism to reorganize the GPU resources. Based on the historically observed distribution of job sizes, we propose a demand-aware fragmentation model that quantifies the fragmentation as a probabilistic measure of a server's inability to accommodate future tasks. The proposed model is formulated as a mixed-integer linear programming (MILP) problem that is solved at each job's checkpoint to determine an optimal set of migrations, thereby continuously reshaping the cluster's resource landscape to maintain high utilization and low queuing delays. The main contributions of this work are as follows,

- We propose a fragmentation-aware model that adjusts resource fragmentation based on job demand distribution, improving the alignment between GPU utilization and workload characteristics.
- We integrate our fragmentation model into an online MILP-based optimization problem to proactively guide the migration of running tasks at their checkpoints.
- We compare our approach with state-of-the-art baselines through event-driven simulations to illustrate the improvement of our model in terms of cluster utilization and job waiting times.

The remainder of this paper is organized as follows. Section 2 reviews related work in resource fragmentation and GPU

cluster scheduling. Section 3 states the main problem, Section 4 details our system model and the mathematical formulation of the optimization problem. Section 5 presents our experimental setup and a comprehensive analysis of the results. Finally, Section 6 concludes the paper.

2. RELATED WORKS

Our work on fragmentation-aware, migration-based scheduling builds upon prior research in cluster management. We review related work in three areas: initial placement scheduling, migration-based resource optimization, and intra-GPU resource management.

Much of the literature on cluster scheduling optimizes job placement. Verma et al. (2015) introduced scoring models to pack tasks efficiently and reduce stranded resources. Grandl et al. (2014) formalized the problem as a multi-dimensional bin packing problem and it is solved via heuristic algorithms. Modern GPU schedulers refine these ideas for fairness or heterogeneity. Ye et al. (2021) applies a long-term fairness metric and lease-based allocation, while Mahajan et al. (2020) uses finish-time fairness and auction-based mechanisms. Wang et al. (2024) and Albahar et al. (2022) address heterogeneity with performance estimators and ML models. Gu et al. (2019) tackles unpredictable job durations using a 2D spatial-temporal scheduler. These systems excel at initial placement but generally lack mechanisms to re-optimize running jobs, allowing fragmentation to accumulate over time.

In order to handle the accumulated fragments, several works utilize live migration technique. Xiao et al. (2018) pioneered time-slicing and migration for DL clusters. Sun et al. (2024) and Ruan et al. (2025) extend this idea for LLM inference and general datacenter workloads, migrating requests or fine-grained “proclets” to balance the load and reclaim resources. ML-guided VM migration has also been used for power management (Belgacem et al., 2023). Our model differs by using a demand-aware fragmentation model to drive proactive migration, continuously reshaping global resource topology to reduce queuing and improve utilization.

We address cluster-level fragmentation across GPUs, distinct from intra-GPU management. Complementary work includes GMLake, which defragments GPU memory via virtual memory stitching (Guo et al., 2024), and AntMan, which dynamically scales per-job GPU memory and compute (Xiao et al., 2020). Other efforts study multitasking policies within a single GPU (Zhao et al., 2024). These systems optimize GPU usage locally, while our model focuses on global GPU allocation.

3. PROBLEM STATEMENT

In the cloud computing environment, enhancing the resource utilization of AI training clusters is a main challenge in resource allocation. However, existing scheduling and migration strategies often lead to GPU resource fragmentation, rendering numerous scattered GPU resources difficult to be utilized effectively, thereby severely impeding the achievement of this goal.

A widely adopted approach relies on heuristic placement algorithms, such as Best Fit. The deficiency of such

algorithms lies in their myopic decision-making nature. When selecting a server for a new task, they focus only on the local optimum of the current placement action (e.g., minimizing the remaining capacity after placement), while completely disregarding the long-term impact of this decision on the future resource topology of the cluster. Consequently, they fail to reserve high-quality, contiguous resource blocks for uncertain future task arrivals.

More advanced strategies guide migrations by setting explicit optimization objectives, such as consolidating tasks onto as few servers as possible. However, this seemingly globally optimal objective can lead to more severe internal fragmentation. Fig. 1 illustrates this deficiency: assume the initial cluster state is (4, 3, 1) as in the Original Load stage, and the task on the third server migrates at a certain point to reserve an empty server as in the Migration stage. A consolidation-oriented migration alters the state to (5, 2, 0). When new tasks requiring (4, 4, 6) GPUs arrive, this new state, due to its internal fragmentation (with remaining capacities of 5 and 3), is unable to accommodate all of them. In contrast, the original state without migration could have easily accepted all new tasks. This demonstrates that merely pursuing the objective of resource consolidation can be counterproductive to the ultimate goal of mitigating fragmentation. Therefore, we need to predict the future demand to perform an appropriate migration.

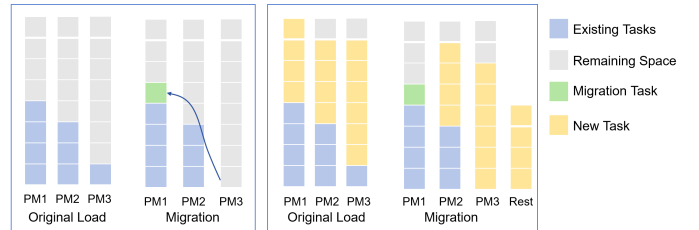


Fig. 1. The counterproductive effect of a resource consolidation strategy, where migrations may prevent new tasks from being placed.

4. MAIN RESULT

4.1 Preliminaries

We model the cluster environment as a discrete-time dynamic system, where a decision epoch, corresponds to the physical time t_c at which a running job triggers a checkpoint. The system comprises a set of homogeneous physical servers, denoted by the set $\mathcal{P} = \{1, 2, \dots, P\}$, where each server $p \in \mathcal{P}$ possesses a fixed GPU card capacity, denoted as C . The GPU card is the indivisible, minimum unit of allocation. The key attributes of an AI training job j are fully described by the tuple $j = (g_j, d_j, a_j, \mathcal{CK}_j)$, where g_j is its GPU requirement, taking a value from the discrete set $S = \{1, 2, 4, 6, 8\}$; d_j is its known and deterministic total runtime; a_j is the job’s arrival time; and $\mathcal{CK}_j$ is the set of checkpoint times determined by the job itself (e.g., save every two hours of execution), which are the only time instant in our model when a migration decision can be triggered.

We utilize the following assumptions throughout the paper.

Assumption 1. The training data resides on shared storage, with only model structure and checkpoints stored locally, making migration time negligible relative to training duration.

Assumption 2. All servers in the cluster are homogeneous, i.e., $C_p = C$, $\forall p \in \mathcal{P}$.

4.2 Fragmentation-aware Optimization Model

Our model is formulated as an MILP problem, which is solved at each migration decision time t_c . The set of decision times $\mathcal{T}$ is defined as all checkpoint times. The objective of the model is to minimize the resource fragmentation across all servers in the cluster after migration, with the optimization scope limited to the set of currently running tasks that have reached the specified checkpoints (hereinafter referred to as the *eligible task set*).

In order to define the fragmentation-aware model, it is necessary to clarify: we focus not on the absolute number k of idle GPUs, but on the impact of remaining resource configurations on the cluster’s ability to accommodate future tasks and the queuing costs within the prediction horizon. Therefore, fragmentation is a relative concept with respect to the task size distribution: if the remaining capacity k can cover certain tasks, it is considered “friendly”; if it cannot cover certain tasks, it is regarded as fragmented resources.

Let the GPU capacity of a single server $C = 8$. First, we clarify the definition of fragmentation relative to the future task demand distribution: let $S \subseteq \{1, 2, \dots, C\}$ be the set of GPU demand sizes of future tasks, and $\pi = \{\pi_s\}_{s \in S}$ be the demand distribution estimated based on historical data (π_s represents the probability that a task’s demand is s , and $\sum_{s \in S} \pi_s = 1$). For a server at time t , its remaining available GPU quantity is $k_t \in \{0, 1, \dots, C\}$. From the perspective of immediate acceptance capability, the coverage set of k_t for the demand set S is defined as $J(k) = \{s \in S \mid s \leq k\}$: if $s \in J(k)$, the server can conditionally accept the task immediately; if $s \notin J(k)$ (i.e., $s > k$), the server has fragmentation relative to such tasks, and the task needs to be queued or routed to other servers, e.g., $k = 4$ has no fragmentation for tasks with $s \leq 4$, but is fragmented for tasks with $s > 4$, shown as 2. Next, based on the arrival distribution, we derive

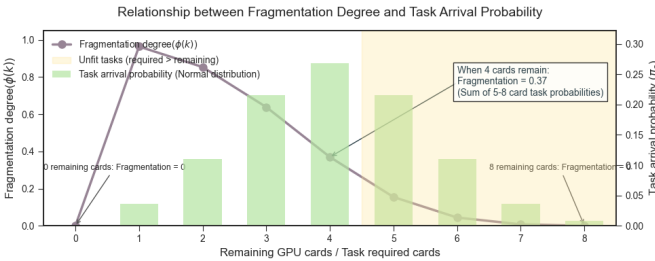


Fig. 2. GPU fragmentation (left axis) and corresponding task arrival distribution (right axis).

the fragmentation cost function $\phi(k)$ (focusing only on blocking risk) to measure the potential mismatch cost of the current remaining capacity k within the time window, which is defined as the expected penalty for blocking risk:

$$\phi(k) = \sum_{\substack{s \in S \\ s > k}} \pi_s \quad (1)$$

Formula 1 represents the probability that a randomly arriving task has a size larger than k (and thus cannot be placed immediately). As k increases, $\phi(k)$ decreases monotonically. Notably, when S consists of discrete values (e.g., $S = \{1, 2, 4, 8\}$), $\phi(k)$ will naturally be the same for certain k values (e.g., $\phi(2) = \phi(3)$, $\phi(4) = \phi(5)$, $\phi(6) = \phi(7)$). In particular, this function satisfies the boundary conditions: $\phi(0) = 0$ and $\phi(8) = 0$, meaning that the fragmentation degree is 0 when the server is idle ($k = 0$) or fully loaded ($k = 8$).

Our objective is to minimize the total fragmentation rate of servers in the cluster after migration. Let L_p denote the number of remaining GPUs on server p at time t , where $L_p \in \{0, 1, \dots, C\}$ and C is the GPU capacity of each server. We aim to minimize the total expected fragmentation cost over the prediction window, while penalizing unnecessary migrations with a constant ϵ :

$$\min_{x_{j,p}} \sum_{p \in \mathcal{P}} \phi(L_p) + \epsilon \cdot \sum_{(j,p)} x_{j,p} \quad (2)$$

Here, $x_{j,p}$ is a migration decision variable (a 0-1 variable indicating migrate task j to server p at the current time), and ϵ only serves to break ties when costs are equal, avoiding meaningless migrations. The key challenge of this objective is that $\phi(L_p)$ is a discrete-nonlinear composite term, ϕ is piecewise constant, and L_p is an integer variable, making direct solution.

In order to address this problem, we linearize $\phi(L_p)$ using one-hot encoding. For each server p at the current time t and each possible remaining capacity $k \in \{0, \dots, C\}$, we introduce one-hot binary variables $f_{p,k}$ that satisfy two conditions: one-hot completeness ($\sum_{k=0}^C f_{p,k} = 1, \forall p$) and the association between remaining value and indicator ($L_p = \sum_{k=0}^C k \cdot f_{p,k}, \forall p$). In GPU clusters with small integer capacity C , these equations combined with the integrality of L_p are sufficient for representation. With $f_{p,k}$, the objective function is linearized as:

$$\min \sum_{p \in \mathcal{P}} \sum_{k=0}^C \phi(k) \cdot f_{p,k} + \epsilon \sum_{(j,p)} x_{j,p} \quad (3)$$

In this linearized form, $\phi(k)$ is a set of constant coefficients determined by the historical distribution π , the objective is linear in $f_{p,k}$, and $x_{j,p}$ remains a 0-1 variable for migration decisions. Together with constraints on capacity balancing, temporal occupation, etc., this forms a solvable MILP problem.

For the calibration of $\phi(k)$, we first use historical logs to estimate the arrival demand histogram:

$$\hat{\pi}_s = \frac{\text{count}(D = s)}{\sum_{u \in S} \text{count}(D = u)} \quad (4)$$

where Laplace smoothing or exponential moving average can be applied for noise resistance. We then compute $\phi(k)$

using the blocking risk formula, and optionally linearly scale it to the interval $[0, 1]$ for easier interpretation and parameter tuning—weights can also be increased for extremely unfriendly configurations (e.g., $k = 1$) based on business preferences. In implementation, the $\phi(k)$ table is a calibrated and scaled numerical version; if the demand distribution changes (e.g., new 6-GPU tasks are introduced), re-estimating π and recalculating $\phi(k)$ will self-consistently reflect the updated relative fragmentation.

Single Migration Constraint Each task in the eligible set is restricted to at most one migration per decision epoch. This is enforced by the constraint:

$$\sum_{\substack{p \in \mathcal{P} \\ p \neq s_j}} x_{j,p} \leq 1, \quad \forall j \in \mathcal{E} \quad (5)$$

Server Capacity Constraint The total GPU allocation on any server p must not exceed its capacity C at any time t within the prediction horizon. The total occupancy, occ_p , is the sum of a fixed component from non-migratable tasks and a dynamic component determined by the migration decisions $x_{j,p}$. The relationship is captured by the constraint $occ_p + L_p = C$, where L_p is an integer variable representing the number of available GPUs on server p at current time.

Linearization Constraints A set of auxiliary constraints is required to enforce the one-hot encoding that binds the integer variable L_p with its binary representation $f_{p,k}$:

$$\sum_{k=0}^C f_{p,k} = 1, \quad \forall p \in \mathcal{P} \quad (6)$$

$$L_p = \sum_{k=0}^C k \cdot f_{p,k}, \quad \forall p \in \mathcal{P} \quad (7)$$

The solution to this MILP formulation, $x_{j,p}^*$, yields an optimal set of migration decisions at current checkpoint, thereby enabling proactive and forward-looking resource defragmentation.

4.3 Algorithm Framework

At each discrete time interval Δt , the system monitors the cluster state and determines whether the current time corresponds to a checkpoint in the decision set $\mathcal{T}$. If $t \in \mathcal{T}$, the algorithm identifies all active tasks that have reached their checkpoint and are eligible for migration. It then constructs and solves a mixed-integer linear optimization problem to minimize the total expected fragmentation across all servers, subject to capacity and single-migration constraints. The optimal migration plan $X^*(t)$ is applied immediately to reallocate resources efficiently. If the current time does not coincide with any checkpoint, the system simply advances by one discrete step Δt and re-evaluates.

This algorithm operates in a rolling manner: at each checkpoint, it re-optimizes task placement based on the current cluster state. Only tasks that have reached their checkpoints are considered for migration, minimizing disruption and unnecessary movements, with a small penalty term ϵ to discourage trivial reallocations.

Algorithm 1 Rolling Checkpoint-Triggered Migration Procedure

Require: Set of running jobs $\mathcal{J}$, where each job $j = (g_j, d_j, a_j, CK_j)$; server set $\mathcal{P} = \{1, \dots, P\}$ with GPU capacity C ; fragmentation cost function $\phi(k)$ and migration penalty ϵ .

Ensure: Migration plan $X^*(t_c)$ at each decision epoch $t_c \in \mathcal{T}$, and the aggregated migration plan X_{all}^* over the horizon.

- 1: Collect all checkpoint times: $\mathcal{T} \leftarrow \bigcup_{j \in \mathcal{J}} CK_j$.
 - 2: Sort $\mathcal{T}$ in ascending order.
 - 3: Initialize $X_{\text{all}}^* \leftarrow \emptyset$.
 - 4: Initialize current time $t \leftarrow t_{\text{now}}$.
 - 5: **while** system is running **do**
 - 6: **if** $t \in \mathcal{T}$ **then**
 - 7: Identify eligible jobs that reach checkpoint t : $\mathcal{E} \leftarrow \{j \in \mathcal{J} \mid t \in CK_j \text{ and } j \text{ is active}\}$.
 - 8: Solve the optimization model at time t .
 - 9: Record $X^*(t)$ into X_{all}^* .
 - 10: **else**
 - 11: Wait until the next time instant.
 - 12: **end if**
 - 13: Update current time $t \leftarrow t + \Delta t$.
 - 14: **end while**
-

5. EXPERIMENTS AND RESULTS

We conduct a systematic evaluation of the proposed scheduling and migration strategies in a multi-machine GPU environment. We perform offline simulations covering varying load intensities and cluster scales. These simulations quantify the average performance and extreme behavior of the algorithms under typical operating conditions, with a focus on key metrics such as resource utilization and queuing delay.

5.1 Simulation Experiments

In our experiments, we built an AI training cluster simulation environment consisting of 50 servers, each equipped with 8 GPUs. The simulation was implemented as a discrete-event model, capturing GPU utilization, job execution time, and migration overhead. New tasks within the 0–100-hour time window followed a Poisson arrival process, and each task was pre-equipped with multiple checkpoints to support flexible checkpoint-based migration. Checkpoints were inserted every 1–3 hours following a uniform random distribution.

To validate the algorithm’s adaptability under heterogeneous workloads, we designed three typical GPU demand distribution scenarios: small-card-dominant (inverse distribution, inv.), medium-card-dominant (normal distribution, norm.), and large-card-dominant (direct distribution, dir.). To demonstrate the superiority of our algorithm, we focused on compute-intensive workloads where job arrivals occasionally exceed available resources, leading to queuing and scheduling delays.

In the offline simulation, the timeline of events can be categorized into four types. The first is the normal running state, during which no new jobs arrive and no existing jobs reach a checkpoint, and the system operates as usual. The second is the checkpoint time (ckpt time), when certain

jobs reach their checkpoints, requiring the system to compute and execute corresponding migration plans. The third is the new job arrival time, when new jobs are submitted to the cluster. To ensure consistency in comparative experiments, we adopt the commonly used best-fit placement strategy, which selects the server that can accommodate the job while leaving the smallest amount of residual resources. The fourth type is the overlapping state, where a checkpoint and a new job arrival occur simultaneously; as shown in Fig. 3, in this case, the system first performs job migration and then proceeds with new job placement. This categorization helps maintain a clear simulation logic and facilitates systematic analysis of migration and placement strategies under different event combinations.

Each configuration was simulated 20 times with different random seeds, and the results were averaged to mitigate stochastic variance.

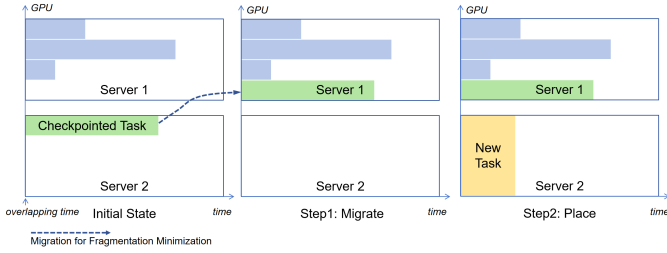


Fig. 3. Handling of overlapping events, where the system first performs migration at checkpoint time and then places the newly arrived task.

5.2 Evaluation Metrics

We evaluate the experimental results using the following metrics.

A fundamental metric for queuing performance is the waiting time of each task i , denoted w_i , which is the interval between the task’s arrival time $t_{a,i}$ and its actual start time $t_{s,i}$, i.e., $w_i = t_{s,i} - t_{a,i}$.

Aggregating this across all tasks in the workload (denoted as set $\mathcal{I}$) yields the mean waiting time, reflecting overall scheduling efficiency $\bar{w} = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} w_i$.

To capture the impact of GPU-demand heterogeneity and fragmentation on system efficiency, we define the *queue card-time* for task i as

$$q_i = g_i \times w_i, \quad (8)$$

where g_i is the number of GPUs required and w_i is the waiting time in queue. Unlike time-based metrics, q_i accounts for both delay and resource demand, thus reflecting the severity of fragmentation—large tasks suffer disproportionately when GPU resources are scattered. The overall metric is given by

$$\bar{Q} = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} q_i, \quad (9)$$

which quantifies the average waiting GPU-time across all tasks and directly measures the system’s responsiveness under heterogeneous GPU-demand distributions.

To measure resource efficiency, we define the average cluster utilization within a time window Q_w , which focuses

on a specific sub-interval of the full simulation period (e.g., 24–48 hours, even if the total simulation runs for 0–100 hours). Formally, this is calculated as $U_w = \frac{Q_w}{(t_e - t_s) \times |\mathcal{P}| \times C}$

where Q_w is the total GPU-time consumed by tasks arriving within the window $[t_s, t_e]$ (with $\mathcal{I}_w$ as the set of tasks active during this window), $|\mathcal{P}|$ is the number of servers, and $C = 8$ (GPU capacity per server).

Additionally, we track *Makespan*, defined as the completion time of the last-finishing task in the workload, which serves as an independent indicator of overall task completion efficiency.

5.3 Results

Our algorithm performs checkpoint-based migration to minimize the fragmentation rate and applies a best-fit strategy for new task placement after migration. To evaluate its effectiveness, we compare it against three baseline schemes: (1) *Inaccurate Prediction*, which introduces a mismatch between the estimated and actual task demand distributions; (2) *Best Fit without Migration (BF)*, which disables migration while retaining best-fit placement; and (3) *Server Consolidation (SC)*, which triggers migration primarily for server consolidation rather than fragmentation reduction, aiming to minimize the number of active servers.

Fig. 4 shows the average GPU utilization over an arbitrarily selected 24-hour window under three demand distributions. Our algorithm consistently attains higher utilization and the performance of *Inaccurate Prediction* further verifies the effectiveness of our fragmentation definition based on the task arrival distribution.

As shown in Fig. 6, across inverse-, normal-, and direct-skewed distributions, our algorithm consistently attains the lowest average waiting time $\bar{w}$ and queue card-time $\bar{Q}$, the highest average utilization $\bar{U}$, and the shortest makespan. The advantages remain stable for both small-card-dense (fine-grained) and large-card-concentrated (coarse-grained) workloads. Compared with load-balancing and first-fit strategies, the fragmentation-aware placement-migration co-design achieves better control of resource waste and faster task response.

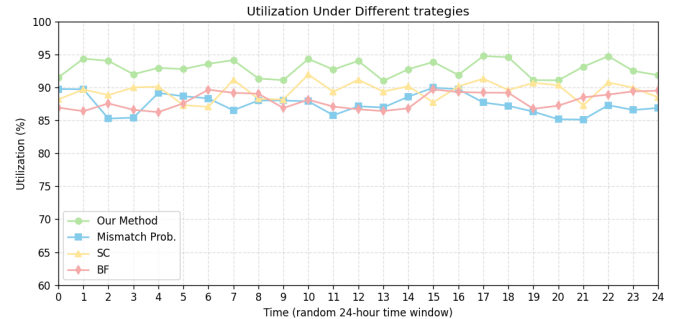


Fig. 4. Average GPU utilization of the cluster within a selected time window under four different methods.

6. CONCLUSION

In this paper, we presented an optimization model based on checkpoint-triggered migration to reduce GPU frag-

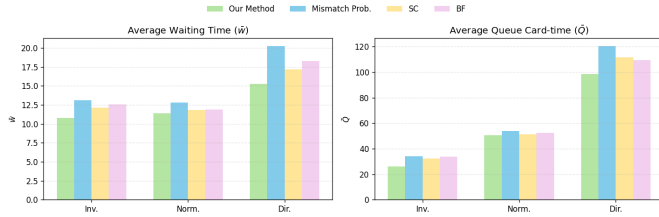


Fig. 5. Average waiting time (left) and average queue card-time (right) under different methods.

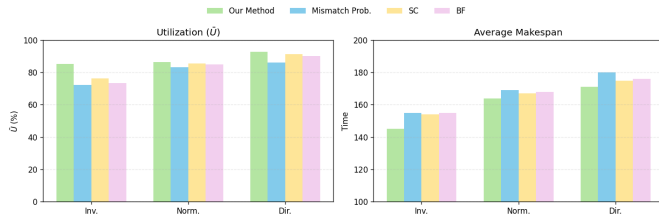


Fig. 6. Average GPU utilization over a 24-hour time window (left) and makespan (right) under different methods.

mentation in AI clusters. We introduced a novel fragmentation metric that adapts to task demand distributions estimated from historical information. It enables the system to better capture the relationship between remaining capacity and workload characteristics. At its core, the proposed approach employs a demand-aware fragmentation model that utilizes historical job distribution data to quantify fragmentation levels and guides a mixed-integer linear programming (MILP) optimization at each checkpoint to continuously reshape the cluster’s resource topology.

Simulation results demonstrate that our model outperforms widely used baseline migration strategies, achieving improvements in both task queuing time and overall GPU utilization. Future work will extend this framework to multi-resource environments, explore heuristic or learning-based solvers for improved scalability, and incorporate more detailed modeling of network topology and migration costs.

Future work will extend this framework to multi-resource environments and explore heuristic or learning-based solvers to improve scalability. We also plan to incorporate load prediction mechanisms so that fragmentation metrics can adapt dynamically to changing workload conditions. In parallel, more detailed modeling of network topology and migration costs will be considered to better capture system dynamics.

REFERENCES

- Albahar, H., Dongare, S., Du, Y., et al. (2022). Sched-Tune: A heterogeneity-aware GPU scheduler for deep learning. In *2022 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, 695–705. IEEE.
- Belgacem, A., Mahmoudi, S., and Ferrag, M.A. (2023). A machine learning model for improving virtual machine migration in cloud computing. *Journal of Supercomputing*.
- Grandl, R., Ananthanarayanan, G., Kandula, S., et al. (2014). Multi-resource packing for cluster schedulers.

- ACM SIGCOMM Computer Communication Review*, 44(4), 455–466.
- Gu, J., Chowdhury, M., Shin, K.G., et al. (2019). Tiresias: A GPU cluster manager for distributed deep learning. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, 485–500.
- Guo, C., Zhang, R., Xu, J., et al. (2024). GMLake: Efficient and transparent GPU memory defragmentation for large-scale DNN training with virtual memory stitching. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 450–466.
- Hadary, O., Marshall, L., Menache, I., and et al (2020). Protean: VM allocation service at scale. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 845–861. virtually.
- Hu, J., Liu, K., Zhang, Y., et al. (2025). A relax-and-round optimization algorithm for online NUMA-aware virtual machine placement. *Expert Systems with Applications*, 271, 126653.
- Mahajan, K., Balasubramanian, A., Singhvi, A., et al. (2020). Themis: Fair and efficient GPU cluster scheduling. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, 289–304.
- Ruan, Z., Li, S., Fan, K., et al. (2025). Quicksand: Harnessing stranded datacenter resources with granular computing. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, 147–165.
- Sun, B., Huang, Z., Zhao, H., et al. (2024). Llumix: Dynamic scheduling for large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 173–191.
- Verma, A., Pedrosa, L., Korupolu, M., et al. (2015). Large-scale cluster management at Google with Borg. In *Proceedings of the tenth european conference on computer systems*, 1–17.
- Wang, Q., Wang, F., and Zheng, X. (2024). Hops: Fine-grained heterogeneous sensing, efficient and fair deep learning cluster scheduling system. In *Proceedings of the 2024 ACM Symposium on Cloud Computing*, 1–17.
- Xiao, W., Bhardwaj, R., Ramjee, R., et al. (2018). Gandiva: Introspective cluster scheduling for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 595–610.
- Xiao, W., Ren, S., Li, Y., et al. (2020). AntMan: Dynamic scaling on GPU clusters for deep learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 533–548.
- Ye, Z., Sun, P., Gao, W., et al. (2021). ASTRAEA: A fair deep learning scheduler for multi-tenant GPU clusters. *IEEE Transactions on Parallel and Distributed Systems*, 33(11), 2781–2793.
- Zhao, X., Wang, H., Huang, A., et al. (2024). Cluster-aware scheduling in multitasking GPUs. *Real-Time Systems*, 60(1), 1–23.